



Project Guide

NOKIA

Hackathon – Project Guide

Introduction	3
1. Project usage	4
1.1. How to launch the base project?	4
1.2. How to upload the solutions?	5
1.3. Project structure	7
1.4. Project configuration	8
2. Tasks overview and evaluation.....	9
2.1. Tasks table	9
2.2. Judge overview.....	10
2.3. Evaluation	10
2.3.1. Testing modes	10
2.3.2. Bonus Points	11
2.4. How are the results displayed?.....	12
2.5. Logs access	13
3. Vivado usage.....	13
3.1. How to create files and add them to the project?	13
3.2. Synthesis and Implementation	13
3.2.1. Failed timing [IMPORTANT].....	14
3.2.2. Project settings [IMPORTANT].....	15
3.3. Adding IP from the IP Catalog	15
3.4. Adding files generated by HDL Coder (MATLAB)	17
4. Debugging.....	18
4.1. Testbench.....	18
4.2. Simulators	18
4.2.1. Vivado Simulator	18
4.2.2. VCS/Verdi	20
4.2.3. QuestaSim or Visualizer.....	23

Introduction

In this instruction, you will find important information that is common to all tasks prepared for FPGA Hackathon 2026.

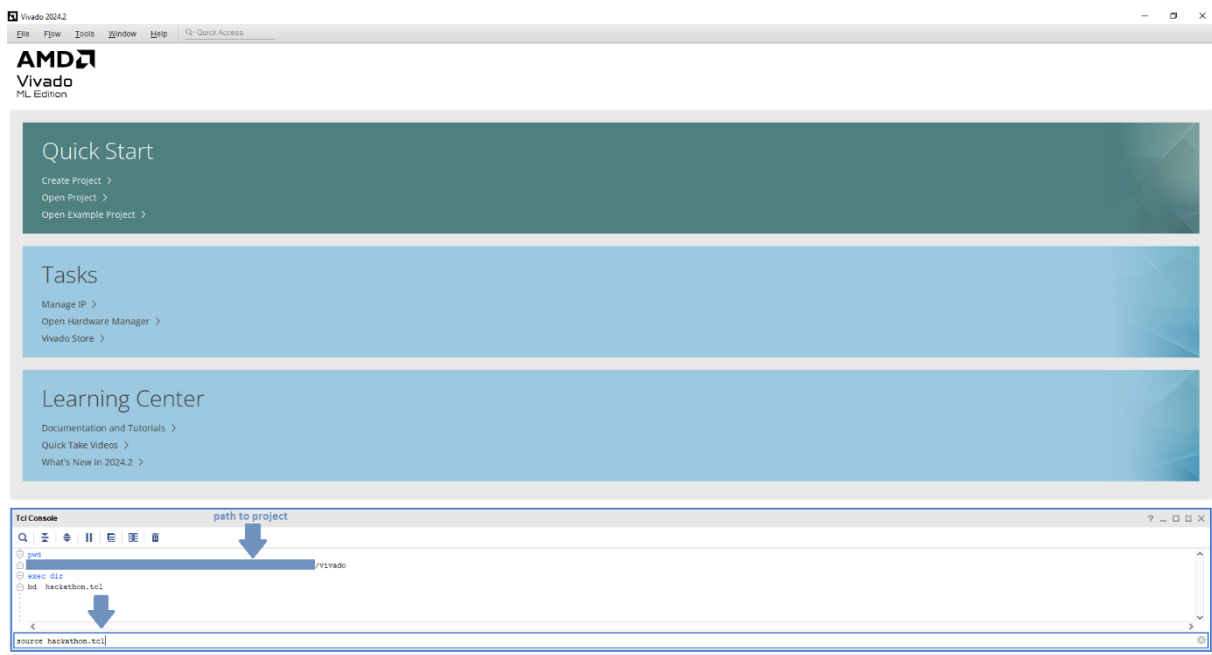
1. Project usage

1.1. How to launch the base project?

You can find instructions on how to download the base project in the **FPGA_Hackathon_2026_User_Manual.pdf**

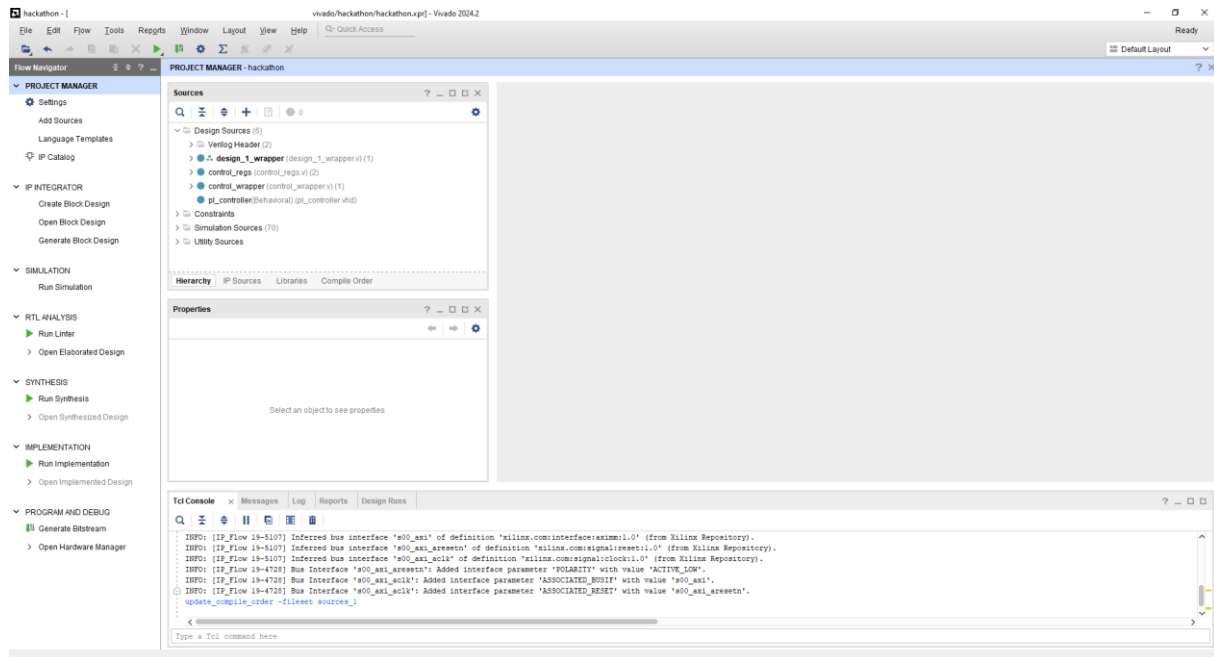
To launch a project, open Vivado 2025.2 and use the Tcl Console to source the *hackathon.tcl* script from *vivado* subdirectory:

```
cd <path_to_project>/vivado/  
source hackathon.tcl
```



Alternatively, if you're already in the *vivado* directory, you can start Vivado and load the project directly.

After sourcing the Tcl script, the software will open the project (it may take a while - be patient). Please be aware that when working on the task solutions with your team members, you will most likely need to cooperate heavily with Git (source control management). Sometimes it may lead to difficulties when opening the Vivado project after pull/merge operations. If such a conflict occurs, try deleting the contents of the *vivado/hackathon* folder, relaunch Vivado, and source the project again using the Tcl file.



On the left-hand side of the main window, you will find the Flow Navigator, which helps you navigate various Vivado features grouped by FPGA development stages. The central panel shows your project sources (note that simulation sources like testbenches are categorized in a different section than design sources). At the bottom, the Tcl Console is opened by default, but you can switch tabs to view messages, logs, reports, or design runs, which may help when debugging errors.

1.2. How to upload the solutions?

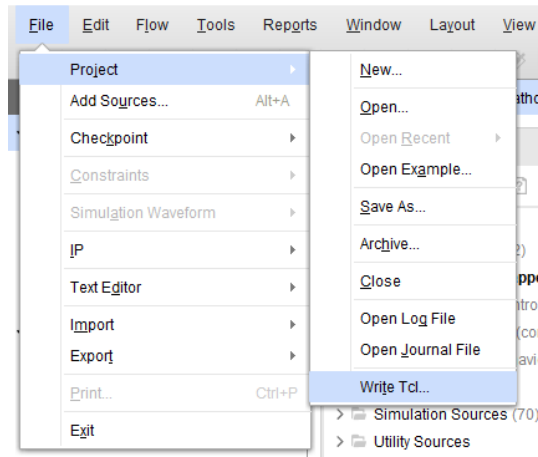
First, you need to create an updated Tcl file. Before doing this, **remove** all *.dcp files generated by Vivado:

```

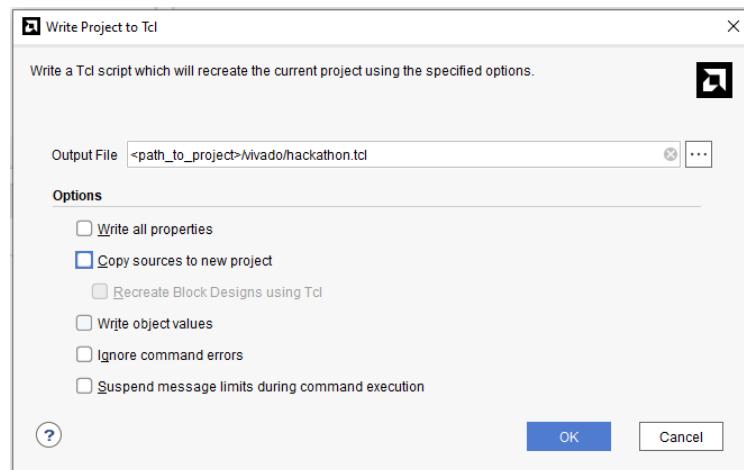
  v Utility Sources (1)
    v utils_1 (1)
      v Design Checkpoint (1)
        design_1_wrapper.dcp

```

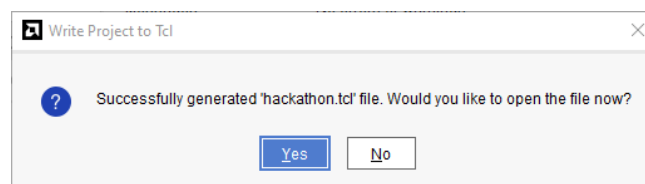
To generate the Tcl file with your current project state, open Vivado and go to *File -> Project -> Write Tcl...*



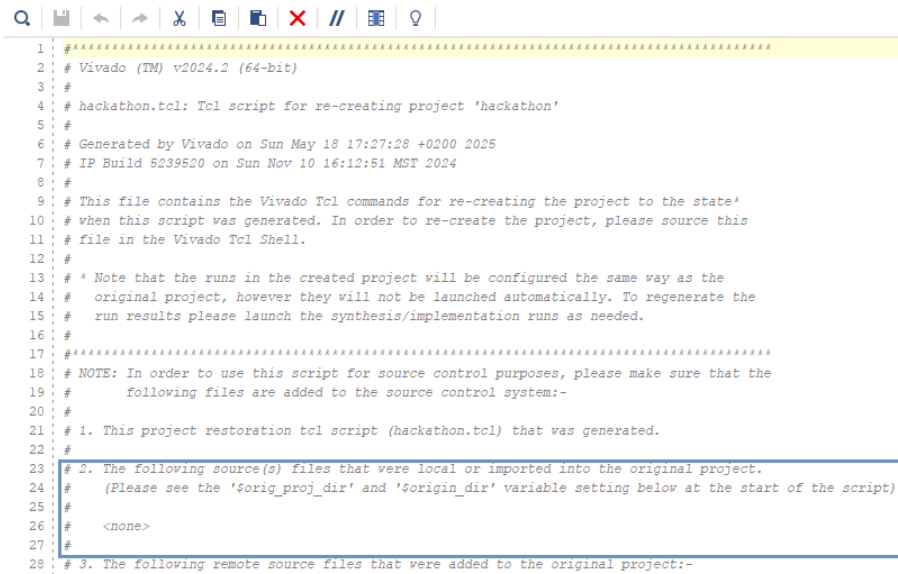
In the new window, as the output file provide the same *hackathon.tcl* file that you used to generate the project initially (double-check the saving directory!). When asked, agree to overwrite the file (**do not change the file name**). Make sure to uncheck the options *Write all properties*, *Copy sources to new project*, and *Recreate Block Designs using Tcl* (as shown below). Select OK and confirm overwriting the file again.



Once the generation is complete, open the generated file.



In the script, check section 2 and ensure that the value is *<none>*. If any file is listed there, it means that a local file is being referenced, and the judge will not be able to access it. In such a case, please update the project so that all required files are properly included. See section 3.1 of this instruction for details on adding files to the project.



```

1 #*****
2 # Vivado (TM) v2024.2 (64-bit)
3 #
4 # hackathon.tcl: Tcl script for re-creating project 'hackathon'
5 #
6 # Generated by Vivado on Sun May 18 17:27:28 +0200 2025
7 # IP Build 5239520 on Sun Nov 10 16:12:51 MST 2024
8 #
9 # This file contains the Vivado Tcl commands for re-creating the project to the state*
10 # when this script was generated. In order to re-create the project, please source this
11 # file in the Vivado Tcl Shell.
12 #
13 # * Note that the runs in the created project will be configured the same way as the
14 #   original project, however they will not be launched automatically. To regenerate the
15 #   run results please launch the synthesis/implementation runs as needed.
16 #
17 #*****
18 # NOTE: In order to use this script for source control purposes, please make sure that the
19 #       following files are added to the source control system:-
20 #
21 # 1. This project restoration tcl script (hackathon.tcl) that was generated.
22 #
23 # 2. The following source(s) files that were local or imported into the original project.
24 #    (Please see the '$orig_proj_dir' and '$origin_dir' variable setting below at the start of the script)
25 #
26 #    <none>
27 #
28 # 3. The following remote source files that were added to the original project:-

```

Add the modified *hackathon.tcl* file, along with all newly created and modified files to a new commit on the main branch. **Do not modify the content of *.gitignore* file.**

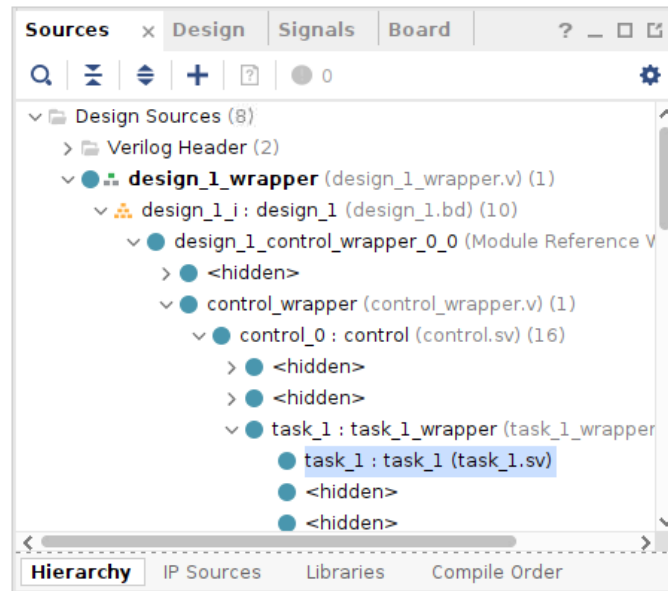
Push your commit to the main branch. If you need any help with source control, you can find a short Git guide in **FPGA_Hackathon_2026_User_Manual.pdf**.

1.3. Project structure

In the repository's main directory, there are three subfolders (src, tb, vivado):

- **siemens** - this folder contains the files needed to complete the task from Siemens.
- **src** – this folder contains the task manager, task configurations, and several control modules. You don't need to modify any of these files except for **task_enabler_pkg.svh** (see section 1.4 for more details).
- **src/task_wrappers** – this folder contains wrapper files responsible for feeding input data to each task and forwarding the results back to the task manager. **Do not modify any files** in this directory.
- **src/tasks/task_x** – each task has its own folder named task_x. In each of these folders, there is one file reserved for your task solution. This file contains the task module definition with its interface and is where you should implement your solution (check section 2.1). You are also allowed to add your own supporting files and instantiate them in the task file, if needed.
- **synopsys** - this folder contains the files needed to complete the task from Synopsys.
- **tb** – this directory contains the testbench. You are encouraged to use it to verify your solution locally before submitting it to the CI pipeline. You may also modify the testbench if needed while debugging. The testbench uses input and output test vector files (*.mem) as reference data, unless the task description specifies otherwise. Section 4.1 explains how to use the provided testbench.
- **vivado** – this folder contains the Tcl script used to generate the project, along with configuration files. Vivado will also generate additional project files in this directory automatically.

Since the project includes encrypted modules, some files may appear as <hidden> in the hierarchy. These correspond to encrypted content, are not meant to be accessed or modified, and can be safely ignored. Below you can find the file hierarchy to help you locate the task file more easily:



<hidden> entities are encrypted files, do not care about them.

1.4. Project configuration

After finishing the implementation of a task, you need to explicitly enable it. Otherwise, the task manager will not report the task as ready for testing - no input data will be sent for that task, and no verification will be performed. To mark a task as completed, update the corresponding parameter in the *src/task_enabler_pkg.svh* file located in the src directory:

```

    Design Sources (6)
    Verilog Header (2)
    task_parameters.svh
    task_enabler_pkg.svh
    design_1_wrapper (design_1_wrapper.v) (1)

```

```

localparam task_enabler enable = '{
    task1 : 1'b1,
    task2 : 1'b0,
    task3 : 1'b0,
    : : :
    : : :
};

```

Set the value to 1'b1 for the task(s) you have completed and want to be tested. Leave the others as 1'b0. Multiple tasks can run simultaneously, and this will be required to maximize your score (see details below).

2. Tasks overview and evaluation

2.1.Tasks table

The order of tasks listed in the project does not reflect their difficulty and has no special meaning. You are free to solve the tasks in any order - start with whichever one you prefer:

Task number	Task name	Task file	Evaluation mode scoring*	Maximum amount of bonus points	Theoretical total points**
1	Shifter	task_1.sv	8	5	13
2	Self-Calibration [HITACHI]	task_2.vhd	30	10	40
3	Morse	task_3.sv	10	5	15
4	Frame Extender [SYNOPSYS]	synopsys/	10	N/A	10
5	Enigma Encryption	task_5.sv	15	5	20
6	Packet Parser [SIEMENS]	siemens/	6	4	10
7	Digital controller for a bionic arm	task_7.sv	16	5	21
8	Software task	task_8.sv	20	N/A	20
9	Cracovian	task_9.sv	15	5	20
10	Memory Mapped	task_10.sv	15	5	20
11	Artifacts	task_11.sv	15	5	20
12	Abacus Romanus	task_12.sv	12	5	17
13	NCO	task_13.sv	12	5	17
14	MCU	task_14.sv	15	5	20
15	LDPC Encoder	task_15.sv	15	5	20
16	AXI Stream Compressor	task_16.sv	12	5	17

* The maximum number of points for each task does not include any bonus points that may be awarded for resource utilization, timing performance, or latency. Detailed information on these criteria can be found in section 2.3.2 and in the individual task descriptions.

** This is a theoretical value due to latency and resource utilization optimization trade-offs that are usually required.

You are allowed to edit only the main task files listed in the table (do not change their filenames or input/output interfaces). However, you may add new submodules and additional files as needed, making sure to place them in the appropriate folders within the *src/tasks/task_x* directory for each task. You can use any language you prefer - SystemVerilog or VHDL.

Tasks 4 and 6 have separate directories: *synopsys/* and */siemens*. Use task descriptions to complete them. (“[TASK 4] Hackathon 7th - SYNOPSYS Frame Extender.pdf” for *synopsys/* and “[TASK 6] Hackathon 7th - SIEMENS - Lab Guide.pdf” and “[TASK 6] Hackathon 7th - SIEMENS - Packet parser.pdf” for *siemens/*).

2.2. Judge overview

Judge is a software application running outside of the board, communicating with the board and verifying task solutions. Judge works in the following flow:

- Connection establishment – the Judge sends a ping message to the board and waits for the pong response, which includes the list of implemented tasks. This step is used to verify which tasks are ready for evaluation. The Judge attempts this step up to three times - if all attempts fail, the evaluation process is aborted. If you notice such scenario in the logs (no pong response), inform the Hackathon's technical team about it – perhaps some extraterrestrials took over your FPGA board!
- Task assessment - based on the list of ready tasks received in the pong message, the Judge proceeds to verify them one by one. The assessment process follows this sequence:
 - Judge sends input data to the task
 - It waits for the FPGA's response for up to 5s
 - It compares the FPGA output with the reference model
 - The score for the task is assigned and stored

There are up to three attempts to obtain the solution for the task. If no result is obtained in that time, the task is scored with **0 points**.

- Report - after verifying all available tasks, the Judge sends the results to the database, updating the score table for the team.

2.3. Evaluation

2.3.1. Testing modes

Each time you trigger a job on Jenkins, your solution will first be tested in **Development mode** for debugging and verification purposes, and immediately afterwards in **Evaluation mode** for final scoring:

- **Development mode** is intended to help you verify and debug your solution. It performs a single set of data per task. The input data in this mode can be randomly generated. In this mode, you have access to logs. This allows the team to verify that their solution is functionally correct. **Results from this mode do not affect your final score.**
- **Evaluation mode** is used to obtain your final score. In this mode, the judge runs several test vectors per task (up to 5), with fixed input data the same for every team. You do not have access to logs in this mode. **Only the latest run will be considered for scoring.**

Each run will test all implemented tasks, and the score will depend on how many of them function correctly, so all tasks must be capable of running simultaneously. More information about the number of test vectors per task and their scoring can be found in the task-specific instructions.

2.3.2. Bonus Points

As part of the evaluation, selected tasks include additional scoring criteria: resource utilization without exceeding the maximum latency. The table below shows which tasks include this extra evaluation aspect - marked with an X in the respective columns. Bonus points for task 6 are explained in task 6 instruction:

Task number	Task name	Resource utilization
1	Shifter	X
2	Hitachi - Measurement Self-Calibration	X
3	Morse	X
4	Synopsys – Frame Extender	
5	Enigma Encryption	X
6	Siemens – Adaptive FIR Filter Bank	
7	Digital controller for a bionic arm	X
8	Software task	
9	Cracovian	X
10	Memory Mapped	X
11	Artifacts	X
12	Abacus Romanus	X
13	NCO	X
14	MCU	X
15	LDPC Encoder	X
16	Axi Stream Compressor	X

Additional points are awarded only if the task is solved correctly (i.e. the maximum number of functional points is obtained):

- The best team receives 5 or 10 additional points
- The second-best team receives 4 or 8 additional points
- The third-best team receives 3 or 6 additional points
- The fourth-best team receives 2 or 4 additional points
- The fifth-best team receives 1 or 2 additional points

2.3.2.1. Additional points for resource utilization

Unless otherwise specified in the task description, resource utilization is evaluated using the following formula:

$$\text{resource_utilization} = \text{CLB_LUTs} + \text{CLB_REGs} + 10 \cdot \text{DSP}$$

The lower the resource_utilization value, the better. For some tasks, the number of bonus points may also depend on the achieved latency value, and bonus points are granted only if

the latency is below the maximum allowed threshold. Maximum latency is specified in each task instruction when applicable.

2.4. How are the results displayed?

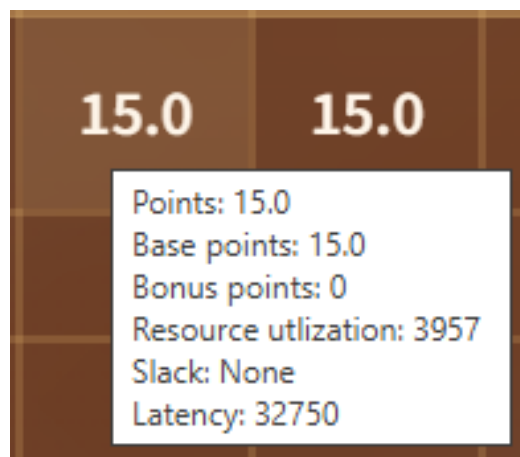
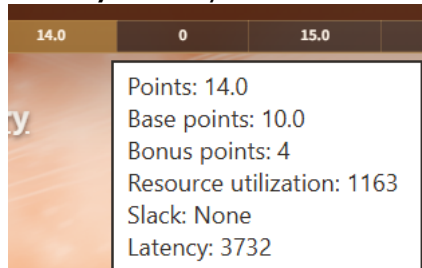
During the hackathon, you will be able to check your team results on the website. At the start of the event, you will obtain a unique link to your team's dashboard.

On the main dashboard page, only your latest runs from Development mode and Evaluation mode will be displayed. There is also a link to a detailed table where all your previous runs (Development Mode only) and results are shown.

You can check detailed information about your results or task specifics by hovering over the respective items in the table. The details include:

- **Points:** total points you earned for the task, divided into:
 - **Base points:** points awarded by the Judge for correct task functionality.
 - **Bonus points:** additional points (if applicable) awarded for resource utilization, and latency (see section 2.3.2 for details):
 - **Resource points:** points calculated based on resource utilization

Latency: latency value measured



You will have access to the general scoreboard with the results of all teams. The link will be shared during the event, but access to the scoreboard will be disabled 3 hours before the event ends.

2.5.Logs access

On the scoreboard website, you will also be able to check logs from your runs in Development mode. There will be a link in the table under the “Logs” column that opens the logs directly in your browser, with an option to download them:

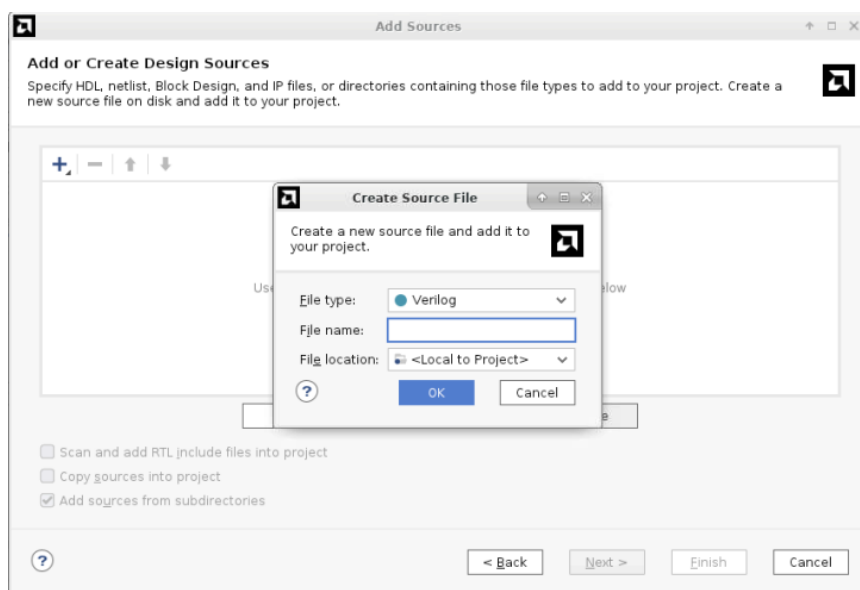
Development Mode									
3.0	5.0	5.0	4.0	4.0	3.0	5.0	4.0	108.0	2026-09-24 07:12:20 Log file

If the link to the log file returns an error, please try again after one minute.

3. Vivado usage

3.1.How to create files and add them to the project?

If you decide to add some files to the project, please use the **ALT+A** shortcut or right- click on *Simulation source/Design sources* in *Sources* tab and choose *Add sources...* . When the window pops up, decide if you want to add a design (RTL) file or a simulation file (testbench). Then choose if you prefer to create a file from scratch (*Create file* option) or just add an existing one. Remember to keep those files in repository folders (design files in appropriate location in *src/tasks/task_x* subdirectory and tests in *tb/tests* subdirectory). **Do not check the box: *Copy sources into project***– instead, make sure your files are stored in the locations mentioned above.



If you add an existing file and have trouble locating it, check the *File of type* field, which may filter files by their extensions. More information is available here: [Creating and Adding Design Sources](#).

3.2.Synthesis and Implementation

We recommend launching a synthesis and implementation of the project before enabling our processing chain.

To launch a synthesis run, use one of the following methods:

- From the Flow Navigator, click Run Synthesis command.
- From the main menu, select Flow > Run Synthesis command.
- In the Design Runs window, right-click the run and select Launch Runs.

The first two options start the active synthesis run. The third option opens the Launch Selected Runs window.

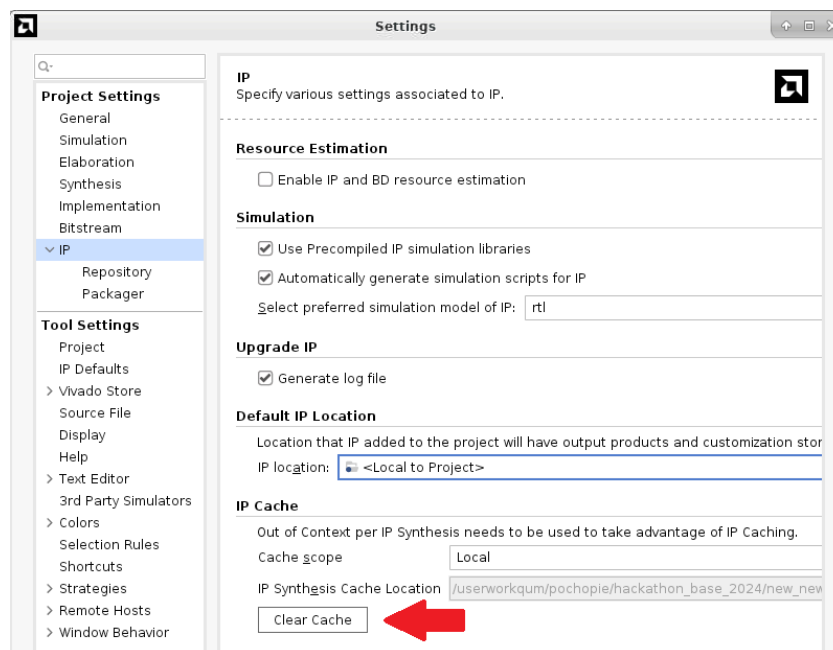
To launch an implementation run, do one of the following methods:

- Select Run Implementation in the Flow Navigator.
- Select Flow > Run Implementation from the main menu.
- Right-click a run in the Design Runs window and select Launch Runs.

To launch a run other than the active one, select the desired run in the Design Runs window before launching.

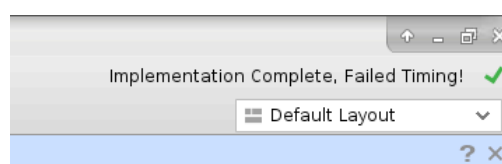
[TIP] If you use VM, we recommend using only 1 job for synthesis (more jobs may slow-down the machine).

[TIP] If changes you made to the design are not reflected in synthesis or implementation, clear the cache: Project Settings -> IP -> "Clear Cache" button at the bottom



3.2.1. Failed timing [IMPORTANT]

There is a chance that synthesis and implementation in Vivado will be completed successfully but timing issues remain. We strongly encourage you to carefully review the synthesis and implementation logs, paying special attention to critical warnings. After synthesis and implementation finish, check the upper right corner:



If you see a warning similar to the one shown above or messages like this:

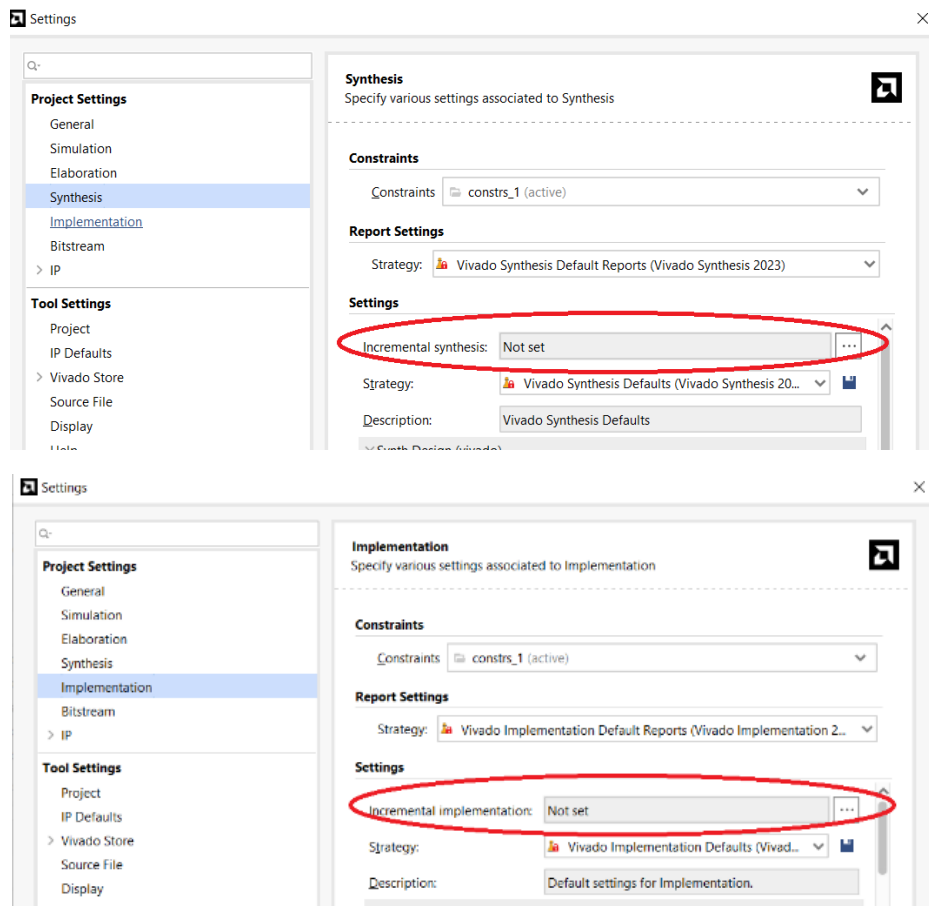
```
1187: INFO: [Timing 38-91] UpdateTimingParams: Speed grade: -2LV, Temperature grade: C, Delay Type: min_max.
1188: INFO: [Timing 38-191] Multithreading enabled for timing update using a maximum of 8 CPUs
1189: CRITICAL WARNING: [Timing 38-282] The design failed to meet the timing requirements. Please see the timing summary report for details on the timing violations.
1190: INFO: [runtcl-4] Executing : report_incremental_reuse -file design_1_wrapper_incremental_reuse_routed.rpt
1191: INFO: [Vivado Tcl 4-1062] Incremental flow is disabled. No incremental reuse info to report.
```

It means your design has timing violations and you need to improve your code to resolve them. Moreover, the Jenkins run will flag it failed (in red) and it will not upload the generated bitfile.

3.2.2. Project settings [IMPORTANT]

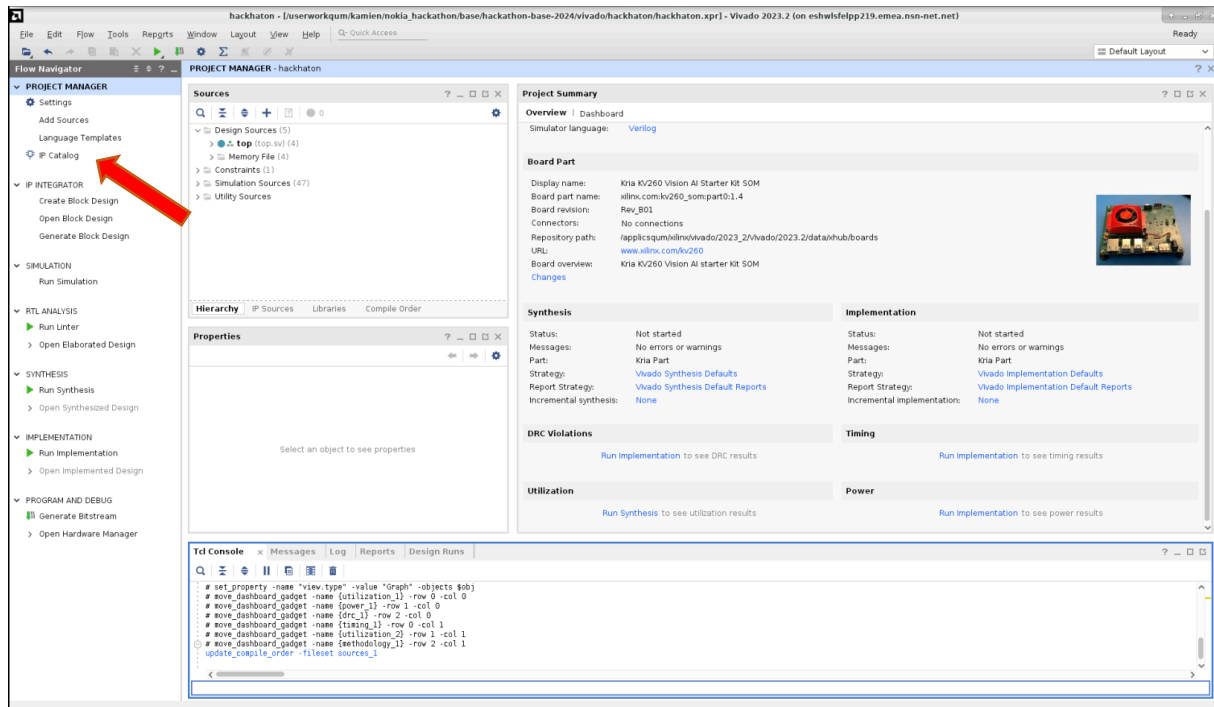
Vivado allows customizing project settings for each processing step: simulation, elaboration, synthesis, implementation, and bitstream generation. Due to the complexity and multiple processing stages in our Hackathon evaluation system, **please do not modify the “Incremental synthesis” and “Incremental implementation” settings**. Leave these settings as **Not set** (see screenshots below).

Changing these options causes additional files to be generated, which are **not supported** by our evaluation chain and will lead to failures when your solutions are tested.



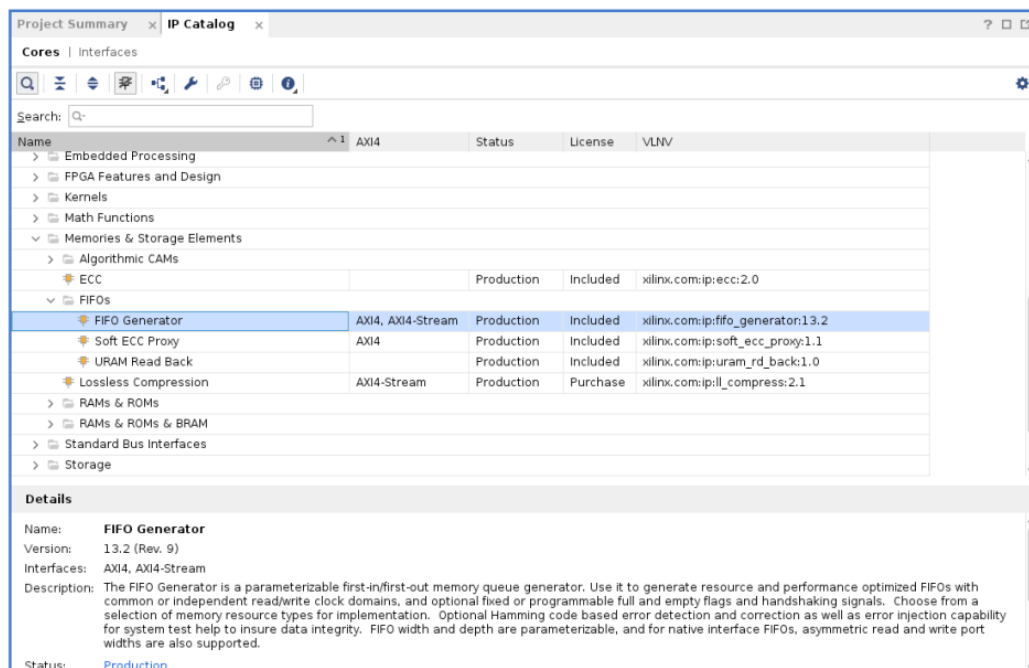
3.3. Adding IP from the IP Catalog

If you want to add an IP from Vivado built-in IP Catalog, navigate to the *Project Manager* section in the Flow Navigator and click on *IP Catalog*. The IP Catalog tab will open.

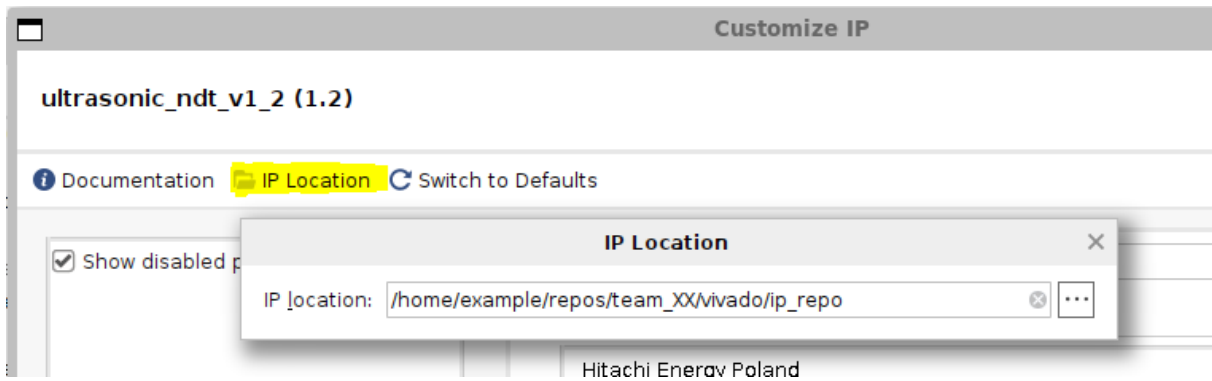


Choose the IP you want to instantiate (you can use the search field to find a specific IP), read the brief description shown at the bottom, then double-click the IP to customize its parameters according to your needs. After finishing the configuration, the IP will appear in your project sources and in the *IP Sources* tab alongside other IPs used in the project.

Save your IP in the *vivado/ip_repo* directory and remember to regenerate the Tcl file. Include all these files when committing your changes.



Set the IP location to *vivado/ip_repo*:

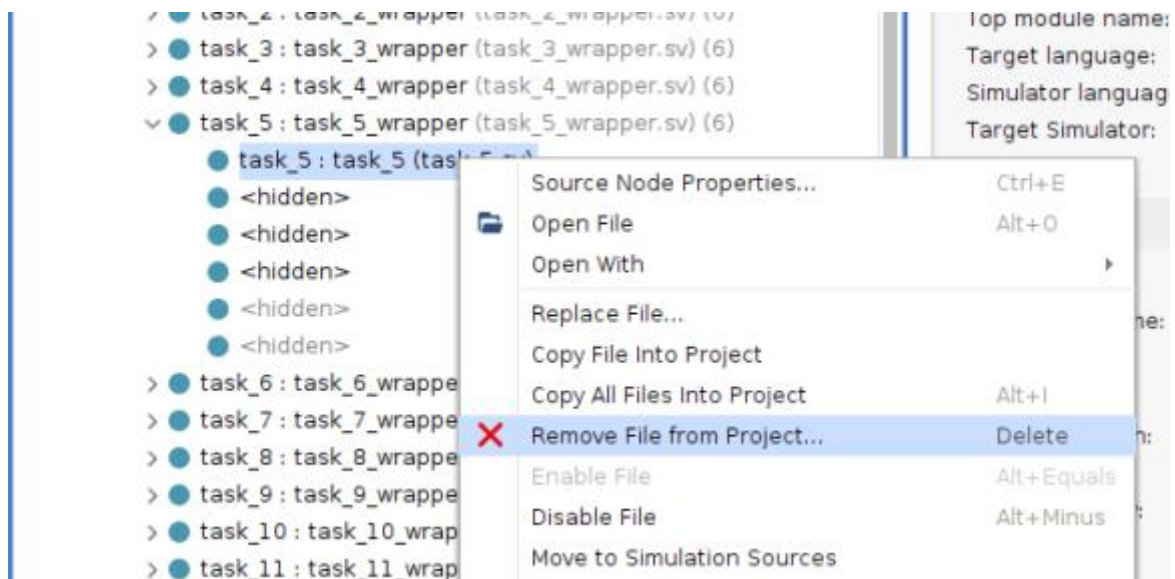


For more details, please refer to AMD's official guide: [Adding IP from the IP Catalog](#).

3.4. Adding files generated by HDL Coder (MATLAB)

If you are using models that have been prepared for you (**do not modify any inputs or outputs**), follow these steps:

- Remove the existing *.sv file from the project and delete it from the task folder - *src/task_**



- Copy the files generated by HDL Coder directly into the corresponding task folder - *src/task_**
- Add these new sources to the project (see chapter 3.1 for instructions)

For more details, refer to MathWorks documentation:

- [Basic HDL Code Generation Workflow](#)
- [Generate HDL Code from Simulink Model](#)
- [Basic HDL Code Generation and FPGA Synthesis from MATLAB - MATLAB & Simulink](#)

4. Debugging

4.1. Testbench

We have prepared a basic testbench file for you - `tb_lite.sv`, located in the `tb` subdirectory. To test a specific task locally, you need to **uncomment the appropriate define** (TASK_N) inside the testbench (others tasks defines should be commented-out – only one define can be active). When you launch a test, a predefined input dataset is sent to your task module, and the module's output will be automatically compared with the corresponding reference data.

```
// SET YOUR TASK HERE!  
//`define TASK_1  
`define TASK_2  
//`define TASK_3  
//`define TASK_4  
//`define TASK_5  
//`define TASK_6  
//`define TASK_7  
//`define TASK_8
```

There are 2 filelists for sim:

-sim_1 with `tb_lite.sv` – recommended to use by you,

-sim_full with `tb_top.sv` – full system simulation, slower, used in case of bugs in our system

4.2. Simulators

There is a possibility to test the created RTL code using several simulators. It is entirely up to you which one you choose. The provided testbench (located in the `tb` subdirectory) is compatible with the most popular simulation tools. If you wish to get more detailed information about simulation workflows in Vivado, we recommend checking the following AMD resources: [Preparing for Simulation](#) and [Logic Simulation](#). Basic simulation instructions and guidance on testbench usage for each supported simulator are provided in the following subchapters.

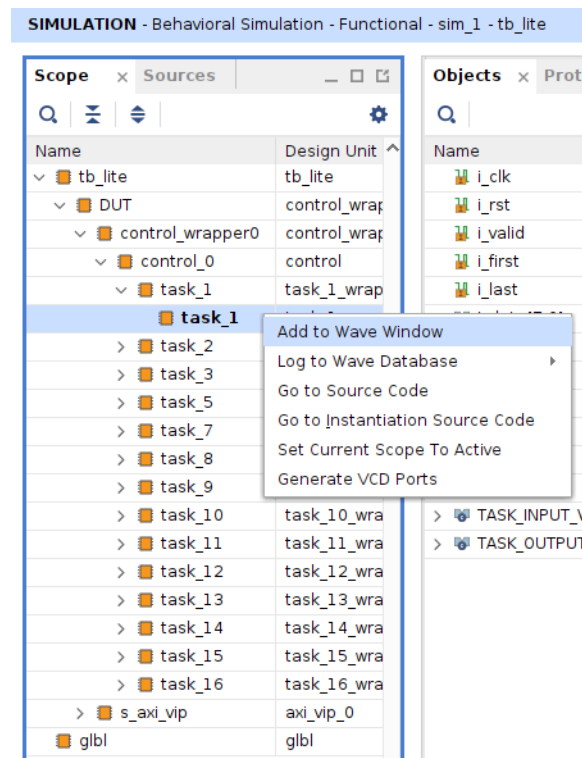
4.2.1. Vivado Simulator

To run a simulation, first verify that you followed the instructions from chapter 2.1 (enabling the correct test). Then, in the Flow Navigator, choose *Run Simulation -> Run Behavioral Simulation* (if you have already synthesized or implemented the design, you may also choose to run post-synthesis or post-implementation simulation).



Once the design is compiled, a waveform window should appear. We have prepared a pre-configured waveform file for you, with relevant task signals already set up. To load it (if not loaded automatically), go to: *File -> Simulation waveform -> Open configuration* and select the file *tb_lite.wcfg* (*tb_behav.wcfg* for full version) located in the *tb* subfolder.

If you want to add more signals to the waveform, navigate to the *Scope* tab and locate the desired module. Then right-click on the module and choose *Add to Wave Window*. The selected signals will be added to the waveform view.

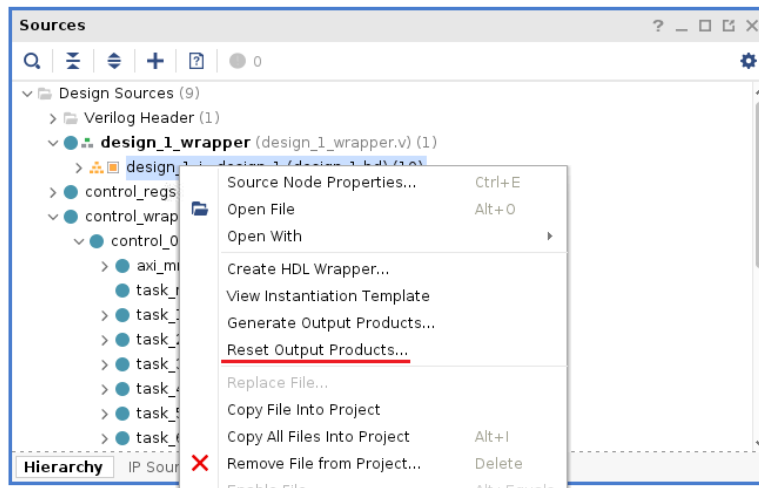


To run the simulation, click the *Run All* button:



[TIP] If the changes you made in the design are not reflected in the simulation (even after relaunching), try the following steps:

- Close the simulation
- Reset output products

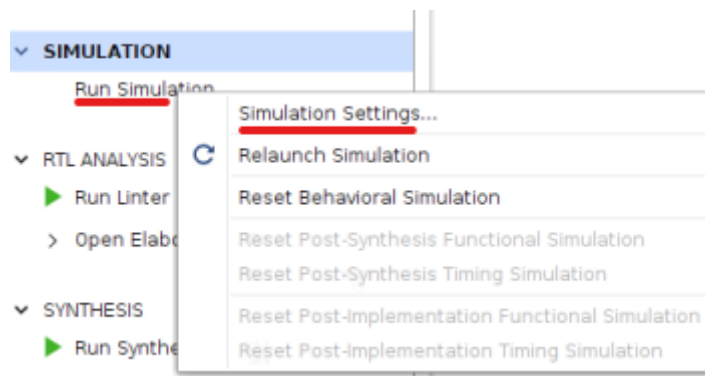


- Run the simulation again

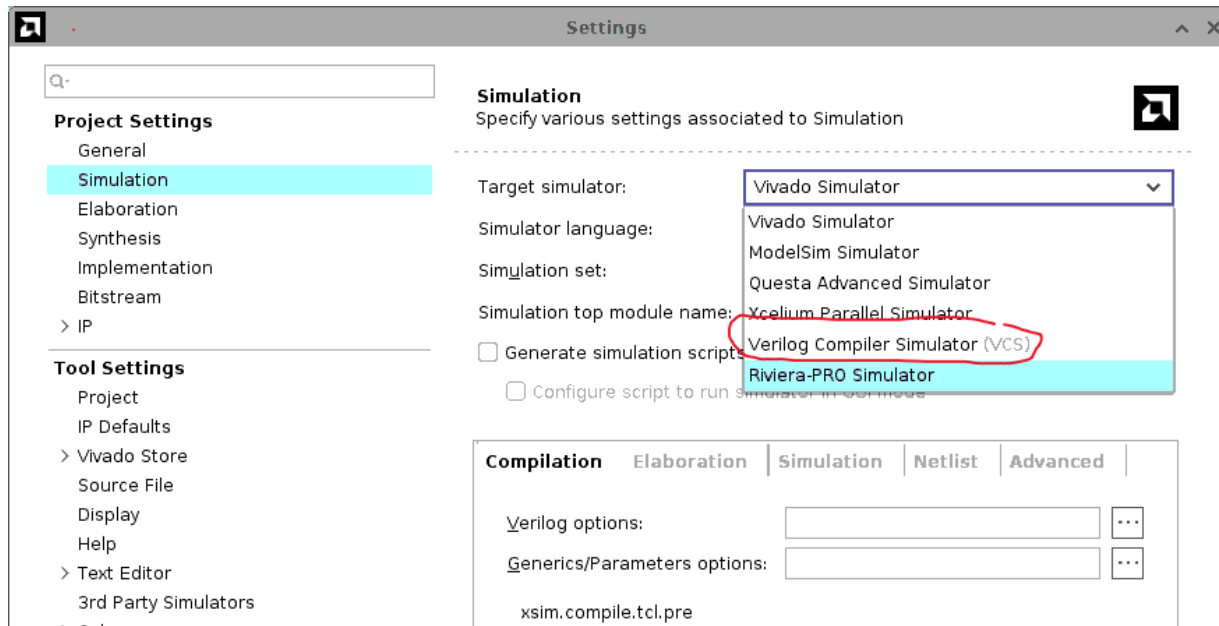
For more information, refer to: [Running the Vivado Simulator](#) or [Analyzing Simulation Waveforms with Vivado Simulator](#).

4.2.2. VCS/Verdi

To run VCS simulator from Vivado gui first right click the SIMULATION/*Run Simulation* tab and go to *Simulation Settings...*

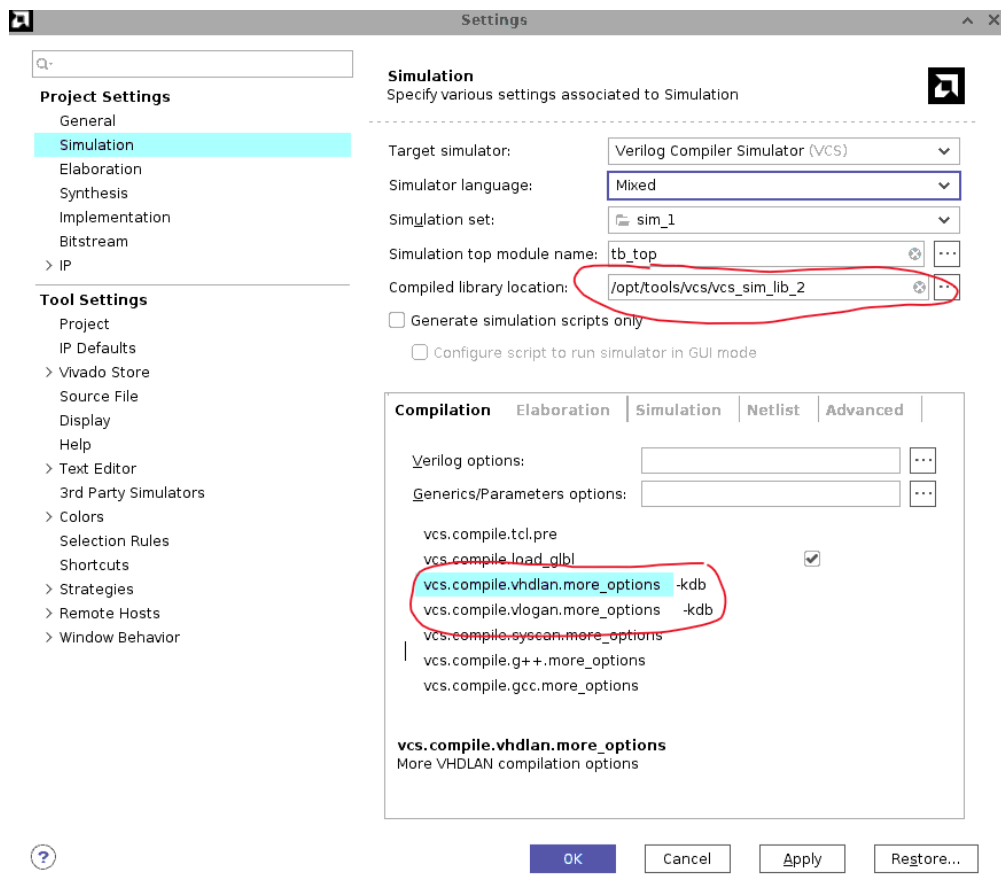


In the Simulation Settings menu select *Target simulator:* to **Verilog Compiler Simulator (VCS)**, and then click *Yes*

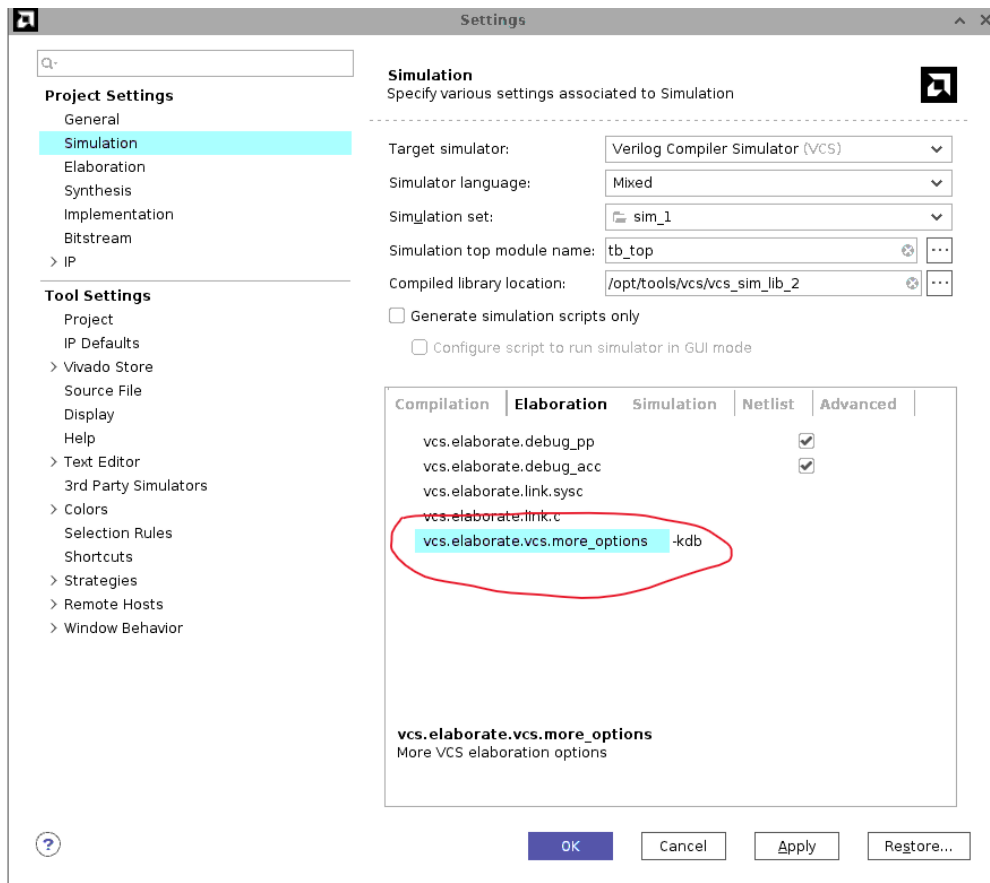


Next, remember to change Compiled library location to precompiled library:

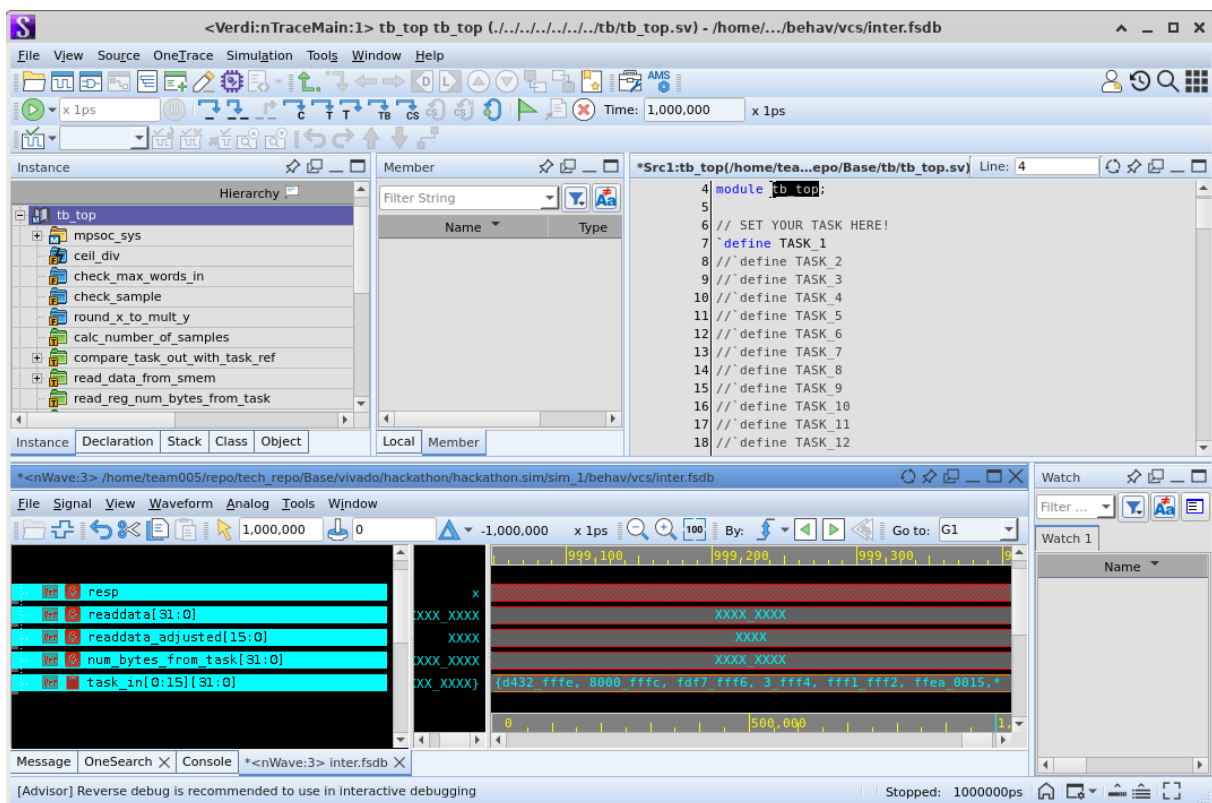
/opt/tools/vcs/vcs_sim_lib_2, and add **-kdb** option to compilation tools (vhdlan and vlogan).



Next, switch to elaborate tab and add **-kdb** option to elaboration and click OK button.



After left-clicking the *Run Simulation* button again, your design will compile and elaborate for VCS and will run the Verdi gui:



4.2.3. QuestaSim or Visualizer

For other siemens tools (Questa Static, Questa Developer, VIP integration) related to task 6 execution, refer to “[TASK 6] Hackathon 7th – SIEMENS – Lab Guide.pdf”.

To run QuestaSim and Visualizer from Makefile, go to /Base/tb/ of your repository in terminal and simply type *make help*, this will show all the available commands for running the batch simulation, Questasim gui and Visualizer gui. There are specific targets to different simulation environments. If you choose *sim_1* you will be running regular flow (make batch, make gui_visualizer etc). If you chose *sim_lite*, you will be running lite flow (make batch_lite, make gui_visualizer_lite etc).

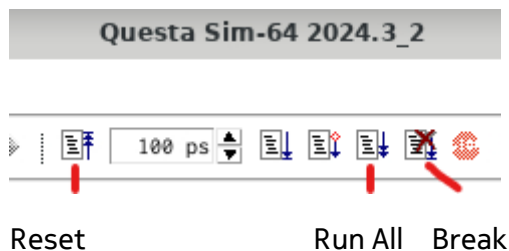
You will only be able to run Makefile based compilation after you run synthesis in vivado first (this is due to generated IP cores and microblaze! It applies to both *sim_1* and *sim_lite*!

Make sure to modify Makefile (/Base/tb/Makefile) whenever you’re adding new files to your project.

Vivado Gui does not support current version of Questasim therefore it won’t be available to run Questasim directly from Vivado.

Helpful Questasim tips:

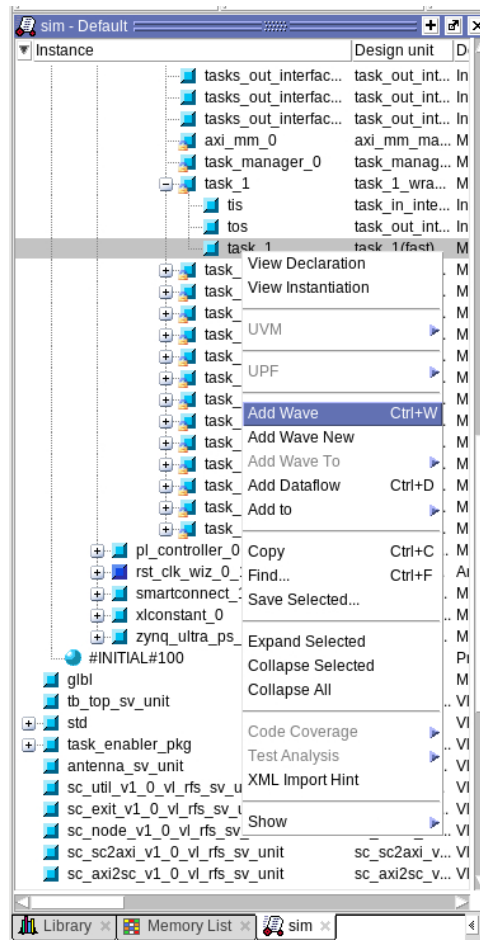
1: Running the simulation:



2: Adding waves to waveform

Go to sim window and navigate to desired hierarchy and then right-click to then select

Add Wave :



3: To maximize windows hit F5, then you can navigate through all the tabs the same size.

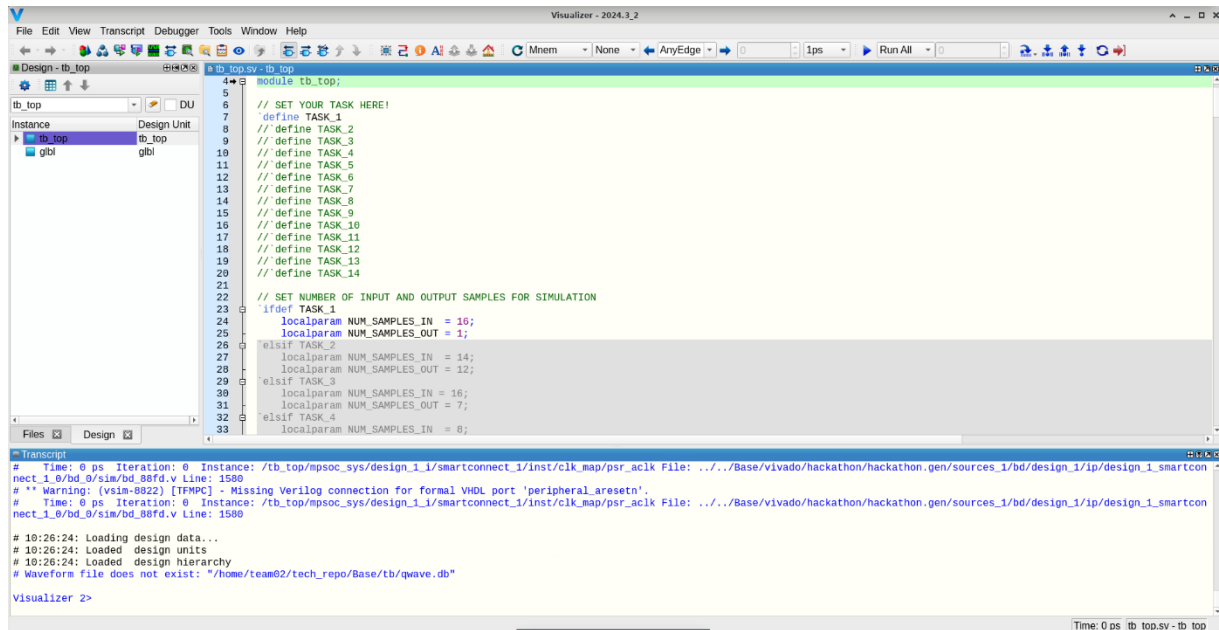
4: Waveform key bindings:

'F' - show full simulation waveform

'C' - Zoom in on the cursor

'I' and 'O' - Zoom in and Zoom out in the center of the screen

Visualizer gui:



To Run simulation click the  button.

To add a waveform, navigate the *Design* tab and right click desired unit and *Add to Wave*:

